Factorisation-Based Multivariate Schemes: Structural Properties and New Constructions

Borja Gomez
{borgomez026}@gmail.com

Abstract

Trapdoor constructions are an active research area in Multivariate Cryptography. The presented work studies trapdoors based on factor decomposition in algebraic structures, with emphasis on polynomial rings over F_p. The main contribution is the formulation of a general property: if an algebraic structure admits a hidden factor decomposition then this property can be used as a trapdoor principle. Based on this approach, two constructions are given: one signature scheme and one encryption scheme.

1 Introduction

Multivariate cryptography usually constructs trapdoors by hiding an easily invertible central map through secret transformations. These central maps use trapdoor function categorised in different families, as later seen. This work considers the construction of schemes using factor decomposition and polynomial factorisation.

One of the first schemes that used a factorisation based trapdoor appeared in 2019, the HIP scheme [1]. Its core map is a bilinear map that multiplies two irreducible polynomials over F_p, followed by a masking linear transformation.

HIP was later broken by Eder [2]. This showed that the original masking layer was insufficient. Later in 2025 a new factorisation-based proposal [3], which proposes a signature algorithm that bases its construction on the concepts of HIP. This study is motivated by the work of the team behind [3], as the author in 2020 devised a similar construction, but didn't publish it. The research from that time is now opened to the public [12], in the form of unfinished drafts, showing prior work on these ideas and concepts.

The objective of this paper is to demonstrate that if an algebraic structure admits a decomposition into factors, then it can serve as the basis of a public-key primitive. This observation is not restricted to the particular polynomial setting of HIP [1] or later related schemes [3]. Any algebraic structure that admits an efficient factor decomposition is a candidate platform for such a construction.

Based on this principle, the paper presents two constructions: a digital signature scheme and an encryption scheme.

1.1 Multivariate Cryptography

In Multivariate Public Key Cryptography (MPKC) public-key schemes work with the following map shape:

$$P = (p_1, \ldots, p_m) : F_q^n \to F_q^m,$$

where the coordinate functions are multivariate polynomials, typically quadratic giving a MQ map, where the basic hardness assumption is the difficulty of solving the system

$$p_1(x) = y_1, \ \ldots, \ p_m(x) = y_m.$$

There are many families that base their constructions on this definition, covered in the standard taxonomy of multivariate schemes [4]. The usual design principle is to start from a structured central map

$$F : F_q^n \to F_q^m$$

for which inversion is efficient with secret information, and then hide that structure through invertible affine or linear transformations. In its standard form, the public map is written as

$$P = T \circ F \circ S,$$

The idea of multivariate cryptography appeared in the Matsumoto–Imai construction, where the central map hides an invertible monomial in an extension field, where is easy to evaluate but only its quadratic public image is exposed [5]. The same philosophy was later developed in the HFE and IP, where the public system is obtained from a hidden polynomial map by composition with secret transformations [6].

From the cryptanalytic side, there are two key components. The first one is direct algebraic solving of the public system, mainly through Gröbner basis methods. Faugère's F4 algorithm is one of the standard tools for this purpose, and the complexity of such attacks is one of the main criteria for parameter selection in multivariate cryptography [7]. This line is important i.e HFE-like systems, where the public equations may be attacked as polynomial systems over F_p without explicitly recovering the hidden map [8].

The second notion is structural reduction to linear algebra problems, especially MinRank-type [10]. In many multivariate schemes, the public equations encode hidden low-rank or low-dimensional relations. When such relations can be extracted, the cryptanalysis may stop being a generic MQ attack and become a structured linear-algebra problem instead.

A secure multivariate construction must resist both sides at once. On one hand, the public map must not admit polynomial system attacks of feasible Gröbner basis complexity. On the other hand, its algebraic shape must not reveal hidden matrix structure that leads to efficient reductions. This relationship between computing an efficient private structure but yet having hard public inversion has been a discussion in the development of the area from its earliest proposals [5, 6, 8].

2 Background

In this study the notation depends heaviliy on concepts from the representation of multilinear maps using tensors. These maps capture the underlying algebraic structure action on two or more elements.

2.1 Quadratic forms and tensor representation

Let F be a field of odd characteristic and let $V = F^n$. In this work, a quadratic form on V is written as

$$q(x) = x^T Q x, \qquad Q \in F^{n \times n}.$$

Equivalently, there exists a bilinear map

$$B : V \times V \to F$$

such that

$$q(x) = B(x, x).$$

Vectorized tensor form If $\mathrm{vec}(Q) \in F^{n^2}$ denotes the vectorization of Q, and

$$x \otimes x = (x_1 x_1, \ldots, x_1 x_n, x_2 x_1, \ldots, x_n x_n)^T \in F^{n^2},$$

then

$$q(x) = \langle \mathrm{vec}(Q),\, x \otimes x \rangle.$$

Hence a quadratic form may be viewed as a linear combination of a $n^2 \times 1$ vector with the tensor $x \otimes x$. Given the public key's quadratic forms, m equations and n variables, the matrix $M \in F_p^{m \times n^2}$ is formed, representing the system of equations as:

$$P = M \cdot (x \otimes x)$$

2.2 Tensor representation of polynomial multiplication

Let p be a prime. Consider two polynomials of degree at most $n-1$ over $F_p[t]$,

$$x(t) = \sum_{i=0}^{n-1} x_i t^i, \qquad y(t) = \sum_{j=0}^{n-1} y_j t^j.$$

Their product is a polynomial of degree at most $2n-2$,

$$x(t)y(t) = \sum_{k=0}^{2n-2} w_k t^k,$$

where each coefficient w_k is given by

$$w_k = \sum_{i+j=k} x_i y_j.$$

This product can be written as a bilinear map from the coefficient vectors of $x(t)$ and $y(t)$ to the coefficient vector of $x(t)y(t)$. For this, define the matrix M that describes polynomial multiplication in tensor form. Its entries are

$$M_{k,\ in+j} = \begin{cases} 1, & \text{if } k = i+j, \\ 0, & \text{otherwise.} \end{cases}$$

Thus M has one row for each output coefficient and one column for each product $x_i y_j$.

If $x \otimes y$ denotes the tensor product of the coefficient vectors of $x(t)$ and $y(t)$, then the action of M gives exactly the coefficient vector of the product polynomial:

$$M \cdot (x \otimes y) = \phi(x(t)y(t)),$$

where ϕ is the map that sends a polynomial to its vector of coefficients.

In this way, polynomial multiplication is represented by the bilinear map

$$F_p^n \times F_p^n \to F_p^{2n-1}, \qquad (x, y) \mapsto x(t)y(t),$$

which induces the linear map

$$F_p^n \otimes F_p^n \to F_p^{2n-1}, \qquad x \otimes y \mapsto M(x \otimes y).$$

This tensor view is used in the rest of this work. It gives a compact and direct way to write polynomial multiplication, and it also extends naturally to multilinear constructions.

2.3 Polynomial factorisation into two components

Let

$$f(t) = \sum_{i=0}^{n} f_i t^i$$

be a polynomial of degree n over F_p. Its factorisation into irreducible factors has the form

$$f(t) = \prod_{i=1}^{r} f_i(t)^{e_i}.$$

The number of distinct irreducible factors is

$$\omega(f) = r,$$

and the number of irreducible factors counted with multiplicity is

$$\Omega(f) = \sum_{i=1}^{r} e_i.$$

Now form the vector of distinct irreducible factors

$$F_B = (f_1(t), \ldots, f_r(t)).$$

The degree of $f(t)$ is the sum of the degrees of the irreducible factors, counted with multiplicity:

$$\deg(f) = \sum_{i=1}^{r} \deg(f_i(t))\, e_i.$$

This gives an integer partition of n,

$$\lambda \vdash n, \qquad \ell(\lambda) = \omega(f),$$

namely

$$\lambda = \big(\deg(f_1(t))e_1, \ldots, \deg(f_r(t))e_r\big).$$

The goal is to decompose $f(t)$ into two factors,

$$f(t) = y_1(t)y_2(t),$$

by selecting a subset of irreducible parts with a target degree k. This gives a subset sum problem on the partition λ.

Let $SS(k, \lambda) = \alpha$ be a subset sum solver that, given target degree k and partition λ, returns a binary vector α. Then α selects the factors that build the first component, while the remaining factors build the second. In this way, one obtains polynomials $y_1(t)$ and $y_2(t)$ such that

$$y_1(t)y_2(t) = f(t).$$

However, the partition λ is sometimes too coarse. Since each irreducible factor $f_i(t)$ appears grouped with its full multiplicity e_i, a solution of target degree

k may fail to exist even when a valid decomposition exists at the polynomial level.

To avoid this problem, the partition must be refined by splitting each multiplicity into separate entries. Define the extended partition

$$\lambda_{\text{ext}} \vdash n, \qquad \ell(\lambda_{\text{ext}}) = \Omega(f),$$

by

$$\lambda_{\text{ext}} = (\underbrace{\deg(f_1(t)), \ldots, \deg(f_1(t))}_{e_1}, \ldots, \underbrace{\deg(f_r(t)), \ldots, \deg(f_r(t))}_{e_r}).$$

In the same way, define the extended factor vector

$$F_{B_{\text{ext}}} = (\underbrace{f_1(t), \ldots, f_1(t)}_{e_1}, \ldots, \underbrace{f_r(t), \ldots, f_r(t)}_{e_r}).$$

Now the target vector for the subset sum has length $\Omega(f)$ instead of $\omega(f)$. This increases the size of the combinatorial search, but it guarantees that a valid decomposition into two factors exists for any target degree compatible with the factorisation.

Given target degree k, solve

$$SS(k, \lambda_{\text{ext}}) = \alpha_{\text{ext}}.$$

Then the selected entries of α_{ext} define the first factor, and the complementary entries define the second. This gives a decomposition

$$f(t) = y_1(t)y_2(t).$$

2.3.1 Factorisation Partition Length

If one wants to estimate the average length of the extended partition for degree-$n-1$ polynomials over F_p, one must consider that this is exactly the same as estimating the average value of $\Omega(f)$ for degree $n-1$ polynomials, which is the number of irreducible factors counted with multiplicity. Indeed, years ago the author asked this question in Math StackExchange [11], seeking for a standard way to estimate the average extended partition length λ_{ext} in relation with the average number of irreducible factors of a monic polynomial.

Following [11], the average length of the extended partition is the same as the average value of $\Omega(f)$ over polynomials of degree $n-1$ in $F_p[t]$. Although the formula in [11] is derived for monic polynomials, the extension to arbitrary polynomials of degree $n-1$ is immediate. Indeed, each monic polynomial of degree $n-1$ has exactly $p-1$ nonzero scalar multiples in $F_p[t]$, all of degree $n-1$, and this scaling does not affect the number of irreducible factors counted with multiplicity.
Therefore the average is unchanged when passing from monic polynomials to all polynomials of degree $n-1$. It follows that

$$\frac{1}{(p-1)p^{n-1}} \sum_{\substack{f \in F_p[t] \\ \deg(f)=n-1}} \Omega(f) = \sum_{m=1}^{n-1} \frac{1}{mp^m} \sum_{r|m} \varphi(r)\, p^{m/r}.$$

2.4 Bilinear Forms

Identify the polynomial multiplication map $F(u, v) = u * v$ with the associated bilinear form $B(u, v) = u * v$. Let $x = (u\|v) \in F_p^{2n}$, where $u, v \in F_p^n$ are the compact vectors representing the two factors.

Define the block matrix

$$S = \begin{pmatrix} S_1 & S_2 \\ S_3 & S_4 \end{pmatrix} \in F_p^{2n \times 2n},$$

and its two block rows

$$L_1 = \begin{pmatrix} S_1 & S_2 \end{pmatrix} \in F_p^{n \times 2n}, \qquad L_2 = \begin{pmatrix} S_3 & S_4 \end{pmatrix} \in F_p^{n \times 2n}.$$

Hence

$$L_1 x = S_1 u + S_2 v, \qquad L_2 x = S_3 u + S_4 v.$$

By bilinearity, the product $B(L_1 x, L_2 x)$ expands as

$$B(L_1 x, L_2 x) = B(S_1 u + S_2 v,\ S_3 u + S_4 v),$$

and therefore

$$B(L_1 x, L_2 x) = B(S_1 u, S_3 u) + B(S_1 u, S_4 v) + B(S_2 v, S_3 u) + B(S_2 v, S_4 v).$$

The four terms correspond respectively to the interactions u-u, u-v, v-u, and v-v.

In the HIP and X-1 schemes, the matrices S_2 and S_3 are zero, while S_1 and S_4 are nonzero invertible matrices. In that case,

$$L_1 x = S_1 u, \qquad L_2 x = S_4 v,$$

so the previous expansion reduces to

$$B(L_1 x, L_2 x) = B(S_1 u, S_4 v).$$

Thus, in HIP and X-1, only the mixed bilinear term survives, whereas the terms $B(S_1 u, S_3 u)$, $B(S_2 v, S_3 u)$, and $B(S_2 v, S_4 v)$ vanish.

The purpose of this study is to extend the family of factorisation-based schemes by allowing the full bilinear expansion, instead of restricting the construction to a single mixed term. In this way, the public equations can be enriched with all admissible bilinear terms arising from the decomposition above.

3 Factorisation-Based Algorithms

This section describes two factorisation-based MPKC schemes, HIP and X-1, describing their common internal structure. In both cases, the central map is the multiplication of two polynomials over $F_p[t]$. This common point is important, since it shows that both schemes follow the same algebraic idea, even if their final purpose is different.

3.1 HIP algorithm

HIP (Hidden Irreducible Polynomials) [1] was presented in 2019 by the same author of the present study. It is based on the bilinear map

$$B(x, y) = x * y,$$

where $x(t), y(t) \in F_p[t]$ are polynomials of degree at most $n - 1$. The specific restriction that makes HIP a KEM construction is that $x(t)$ and $y(t)$ are required to be irreducible over F_p.

3.1.1 Construction

Given F_p and $m = 2n - 1$, define two variable sets $x, y \in F_p^n$ and their compact vector

$$z = (x \| y) \in F_p^{2n}.$$

The rest of the construction is as follows.

First define the multiplication matrix

$$M \in F_p^{m \times n^2},$$

as in Subsection 2.2, and let

$$T \in F_p^{m \times m}.$$

Now define

$$S = \begin{pmatrix} I_n & 0 \\ 0 & I_n \end{pmatrix} \in F_p^{2n \times 2n}.$$

Its two block rows are

$$L_1 = \begin{pmatrix} I_n & 0 \end{pmatrix} \in F_p^{n \times 2n}, \qquad L_2 = \begin{pmatrix} 0 & I_n \end{pmatrix} \in F_p^{n \times 2n}.$$

The public key is then

$$P = T \cdot M \cdot (L_1 \otimes L_2) \cdot (z \otimes z).$$

Since L_1 selects the first half of z and L_2 selects the second half, the central map is exactly the product of the two input polynomials. HIP keeps only the mixed bilinear term and does not introduce extra x-x or y-y terms.

Remarks A main weakness appears at this point. Here S is the identity matrix, and the central map F is public. Therefore the secret part is reduced to the outer linear map T. Since F is known, an attacker can recover T by solving a linear system [2]. This makes the hiding layer too weak.

The HIP paper remarks that taking matrices $S_i \neq I_n$ would destroy the use of irreducible polynomials. The reason is simple. If we replace the input by

$$S \cdot z = z',$$

then the new vector z' no longer keeps the original split into two degree $n-1$ polynomials. After the change of variables, the two resulting polynomials may no longer be irreducible. Thus, the irreducibility condition is not stable under a general linear transformation of the input.

3.2 X-1 algorithm

In 2025, the team behind [3] introduced the X-1 algorithm. It is also built from the bilinear form

$$B(x, y) = x * y,$$

and it gives a digital signature construction.

X-1 keeps the same central idea as HIP, the multiplication of two polynomials over $F_p[t]$, but changes several important parts of the scheme. First, the matrix S_1 is taken invertible and is no longer the identity. Second, the input polynomials $x(t), y(t)$ are not required to be irreducible, any two polynomials may be used. This makes the final recovery step rely on the factorisation of their product, using the decomposition described in Subsection 2.3.

Another important difference is the linear map T. In X-1, this matrix has size $m \times k$, so the number of public equations may be smaller than the full output dimension. Therefore the scheme does not require $m = k = 2n - 1$, and some equations may be deleted from the public key.

3.2.1 Construction

Given F_p and $m \leq k = 2n - 1$, define two variable sets $x, y \in F_p^n$ and their compact vector

$$z = (x \| y) \in F_p^{2n}.$$

The rest of the construction is as follows.

First define the multiplication matrix

$$M \in F_p^{k \times n^2},$$

as in Subsection 2.2, and let

$$T \in F_p^{m \times k}.$$

Now define

$$S = \begin{pmatrix} S_1 & 0 \\ 0 & S_1 \end{pmatrix} \in F_p^{2n \times 2n},$$

where S_1 is invertible. Its two block rows are

$$L_1 = \begin{pmatrix} S_1 & 0 \end{pmatrix} \in F_p^{n \times 2n}, \qquad L_2 = \begin{pmatrix} 0 & S_1 \end{pmatrix} \in F_p^{n \times 2n}.$$

The public key is then

$$P = T \cdot M \cdot (L_1 \otimes L_2) \cdot (z \otimes z).$$

As in HIP, the central map still keeps only the mixed bilinear term. However, the variables are first transformed by the invertible matrix S_1. From this point of view, X-1 can be seen as a natural extension of HIP. It removes the irreducibility restriction, introduces a nontrivial input transformation, and allows a rectangular matrix T. These changes make the scheme more flexible and more attractive for signature design.

3.2.2 Signature generation

The signature process is implemented in [3]. Note that step 3 restricts the image of the central map to be the product of two $n-1$ polynomials. This is true when the image has degree $2n-2$.

1. Let $h = (h_1, \cdots, h_m) \in F_p^m$ be the hash of the message M
2. Compute a random solution
$$y = (y_1, y_2, \ldots, y_k) \in F_q^k$$
to the linear system
$$Ty = h,$$
with
$$y_k \neq 0.$$
3. Construct the polynomial
$$f(z) = y_1 + y_2 z + \cdots + y_k z^{k-1}.$$
4. Factor
$$f(z) = f_1(z) f_2(z),$$
where
$$\deg f_1 = \deg f_2 = n-1.$$
If such a factorisation exists, continue. Otherwise, go back to Step 1 and repeat.
5. Write
$$f_1(z) = x_1 + x_2 z + \cdots + x_n z^{n-1}, \qquad f_2(z) = x_{n+1} + x_{n+2} z + \cdots + x_{2n} z^{n-1}.$$
Set
$$x' = (x_1, x_2, \ldots, x_n), \qquad x'' = (x_{n+1}, x_{n+2}, \ldots, x_{2n}).$$
Compute
$$u = (u_1, u_2, \ldots, u_n), \qquad v = (v_1, v_2, \ldots, v_n) \in F_q^n$$
by solving the linear systems
$$S_1 u = x', \qquad S_1 v = x''.$$
Then set
$$u = \lambda_1 u, \qquad v = \lambda_2 v,$$
for some nonzero scalars
$$\lambda_1, \lambda_2 \in F_q,$$
so that the leftmost nonzero entry of both u and v is equal to 1.
6. The signature of M is
$$u \quad \text{if } u < v, \qquad \text{and } v \quad \text{if } v \leq u.$$

4 New Factorisation Based Algorithms

From now only the central polynomial

$$F(x, y) = B(x, y)$$

has been studied for HIP and X-1 algorithms. In this section new algorithms are presented that work with the central polynomial

$$F(x, y) = B(x, x) + B(x, y) + B(y, x) + B(y, y)$$

that is formed by the sums of all possible bilinear terms.
The major contribution here is to be able to invert to generate signature points or plaintext encipher/decipher.
The central map introduces more complex terms $x_i x_j$ and $y_i y_j$ to the already existant $x_i y_j$ monomials.

4.1 Universal decomposition property

The following property applies to multidimensional maps built from d tensor components $x^{\otimes d}$. The main idea is simple: the output of the map must admit a decomposition into d distinct parts in the chosen algebraic structure. This gives a common way to describe factorisation type constructions.

4.1.1 Quadratic case

Consider the multiplication map $F(a, b) = y$. Assume that the output y can be decomposed into two parts y_1, y_2 in the underlying algebraic structure.

Set the compact vector $x = (a\|b)$. Let $L_1, L_2 \in F_p^{n\times 2n}$. Then

$$y = B(L_1 x, L_2 x),$$

that is,

$$y = B(S_1 a + S_2 b,\ S_3 a + S_4 b).$$

Now stack the two linear maps as

$$S = \begin{pmatrix} L_1 \\ L_2 \end{pmatrix}.$$

For invertible S then

$$S^{-1}(y_1\|y_2) = x = (a\|b).$$

This gives the recovery of the compact input from the decomposed output.

For dimension counting, suppose the underlying structure is the finite field F_{q^n}. Then the map $F(a, b) = y$ gives n equations in $2n$ variables. In contrast, if we work with polynomial multiplication in $F_p[t]$ without reduction modulo an irreducible polynomial, then the product has degree up to $2n - 2$. Hence the map gives $2n - 1$ equations in $2n$ variables.

The chosen algebraic structure changes the number of equations, and therefore changes the shape of the public system and its possible hardness.

4.1.2 Cubic case

The same idea extends to degree 3. Now the compact vector is

$$x = (a\|b\|c),$$

and we use three linear maps $L_1, L_2, L_3 \in F_p^{n\times 3n}$. The output is written as

$$y = B(L_1x, L_2x, L_3x),$$

that is,

$$y = B(S_1a + S_2b + S_3c,\ S_4a + S_5b + S_6c,\ S_7a + S_8b + S_9c).$$

Now stack the three maps as

$$S = \begin{pmatrix} L_1 \\ L_2 \\ L_3 \end{pmatrix}.$$

If y decomposes into three parts $(y_1\|y_2\|y_3)$ and S is invertible, then

$$S^{-1}(y_1\|y_2\|y_3) = x = (a\|b\|c).$$

If the chosen structure is $F_p[t]$, then the product of three degree-$(n-1)$ polynomials has degree up to $3n-3$. Therefore the map gives $3n-2$ equations in $3n$ variables.

More generally, for tensor dimension d, polynomial multiplication in $F_p[t]$ gives

$$d(n-1)+1$$

equations in dn variables.

The trilinear case is not studied in this work. A main reason is that cubic schemes usually require larger public keys, which makes both construction and analysis heavier.

Remarks This property is not limited to the examples above. Other algebraic structures may also provide valid decompositions of the output. This can be useful for the design of new schemes, and it may open new research lines on how the chosen structure affects efficiency, public key size, and algebraic security.

4.2 Facto-DSA

Facto-DSA is a signature construction built from the same core factorisation idea, but unlike earlier schemes it allows all bilinear monomial terms to appear in the public key. In this sense, the scheme keeps the same general principle while enlarging the central map. Our experiments suggest that this richer structure gives higher attack complexity, at least from the point of view of Gröbner basis solving.

4.2.1 Construction

Given F_p and $m \leq k = 2n - 1$, define two variable sets $x, y \in F_p^n$ and their compact vector

$$z = (x \| y) \in F_p^{2n}.$$

The rest of the construction is the following.

First, define the multiplication matrix

$$M \in F_p^{k \times n^2},$$

as in Subsection 2.2, and let

$$T \in F_p^{m \times k}.$$

Now define the block matrix

$$S = \begin{pmatrix} S_1 & S_2 \\ S_3 & S_4 \end{pmatrix} \in F_p^{2n \times 2n},$$

together with its two block rows

$$L_1 = \begin{pmatrix} S_1 & S_2 \end{pmatrix} \in F_p^{n \times 2n}, \qquad L_2 = \begin{pmatrix} S_3 & S_4 \end{pmatrix} \in F_p^{n \times 2n}.$$

The public key is then given by

$$P = T \cdot M \cdot (L_1 \otimes L_2) \cdot (z \otimes z).$$

The main point is that the central map now contains the full bilinear expansion. Indeed,

$$F(x, y) = B(L_1 z, L_2 z) = B(S_1 x + S_2 y,\ S_3 x + S_4 y),$$

as discussed in Subsection 2.4. Hence the map is not restricted to a single mixed product. Instead, it includes all bilinear contributions induced by the choice of S.

After expansion, the central map contains the terms

$$B(S_1 x, S_3 x), \qquad B(S_1 x, S_4 y), \qquad B(S_2 y, S_3 x), \qquad B(S_2 y, S_4 y).$$

Therefore, the construction may contain x-x, x-y, y-x, and y-y interactions at the same time. This is the main difference with more restricted constructions such as HIP or X-1, where only one mixed term is kept.

4.2.2 Signature generation & verification

The polynomial $y(t)$ associated with the image point has degree at most $2n-2$. A valid signature is obtained by decomposing $y(t)$ into two factors of suitable degrees and then applying the inverse linear map.

- Alice selects a message value $m \in F_p^m$ and a salt s.
- Alice computes the hash value

$$h = H(m\|s).$$

- Alice solves the linear system

$$T \cdot y = h,$$

  where y is a vector of length $k = 2n-1$. Viewed as a polynomial, $y(t)$ has degree at most $2n-2$.
- Alice factors $y(t) \in F_p[t]$ into irreducible factors.
- Alice invokes the subset sum solver on the corresponding partition data, with target degree $d \leq n-1$, as explained in Subsection 2.3.
- Each valid subset sum solution gives a candidate decomposition

$$y(t) = y_1(t)y_2(t),$$

  with $\deg(y_1)$ or $\deg(y_2) \leq d$. If no solution exists for the chosen value of d, Alice decreases d and repeats.
- If d reaches 0, repeat from step 1.
- Once a valid pair (y_1, y_2) is found, Alice computes

$$z = S^{-1} \cdot (y_1\|y_2).$$

  This gives a valid signature point. Alice sends (z, m, s) to Bob.
- Bob verifies the signature by checking that

$$P(z) = H(m\|s).$$

4.3 EDF

EDF (Equal Degree Factorisation) is an algorithm that aims to encipher and decipher plaintext using factorisation as its core trapdoor. The principal idea is to multiply two monic polynomials $x(t), y(t) \in F_p[t]$ then factor them using the extended partition obtained from the irreducible degree decomposition λ_{ext} as seen in subsection 2.3, check "tradeoffs".

4.4 Comparison to other schemes

EDF follows the same main idea as HIP and X-1, but adds one extra output dimension and one linear layer. This lets the scheme represent, in internal form, the multiplication of two monic polynomials of degree n.

This changes the final decomposition step into an equal-degree factorisation problem, which can be seen as a partition problem, that is, a special case inside subset sum. In addition, EDF gives a system with $m = 2n$ equations in $2n$ variables, and all monomial combinations may appear in the public equations.

4.4.1 Theory

In HIP and X-1, the core map multiplies two polynomials of degree $n-1$. EDF keeps the same core idea, but adds a linear correction so that two monic degree-n polynomials are handled internally.

Let

$$\hat{x}(t) = t^n + x(t), \qquad \hat{y}(t) = t^n + y(t),$$

where

$$x(t) = \sum_{i=0}^{n-1} x_i t^i, \qquad y(t) = \sum_{i=0}^{n-1} y_i t^i.$$

Then

$$\hat{x}(t)\hat{y}(t) = t^{2n} + t^n\big(x(t) + y(t)\big) + x(t)y(t).$$

Hence the quadratic part is still the product $x(t)y(t)$ of two degree $n-1$ polynomials, while the term $t^n(x(t) + y(t))$ is linear in the coefficients of $x(t)$ and $y(t)$.

Therefore EDF may be viewed as the map

$$\overline{F}(x, y) = x(t)y(t) + t^n\big(x(t) + y(t)\big),$$

which has degree at most $2n-1$. Adding back the leading monic term t^{2n} gives

$$t^{2n} + \overline{F}(x, y) = \hat{x}(t)\hat{y}(t),$$

so the output is a monic polynomial of degree $2n$ admitting a decomposition into two monic factors of degree n.

4.4.2 Construction

Given F_p and $m = 2n$, define two variable sets $x, y \in F_p^n$ and their compact vector

$$z = (x \| y) \in F_p^{2n}.$$

The rest of the construction is as follows.

First, note that the multiplication matrix M now has one extra zero row. This is because here the output dimension is $m = 2n$, instead of the usual case $m = 2n - 1$. Thus,

$$M \in F_p^{m \times n^2},$$

as defined in Subsection 2.2. Also define

$$T \in F_p^{m \times m}.$$

Next, introduce the block matrix

$$S = \begin{pmatrix} S_1 & S_2 \\ S_3 & S_4 \end{pmatrix} \in F_p^{2n \times 2n},$$

and its two block rows

$$L_1 = \begin{pmatrix} S_1 & S_2 \end{pmatrix} \in F_p^{n \times 2n}, \qquad L_2 = \begin{pmatrix} S_3 & S_4 \end{pmatrix} \in F_p^{n \times 2n}.$$

We also define the linear map

$$L = \begin{pmatrix} 0 & 0 \\ I_n & I_n \end{pmatrix} \in F_p^{2n \times 2n}.$$

When the input vector is written as $z = (x \| y)$ with $x, y \in F_p^n$, the vector Lz represents the coefficient vector of the polynomial

$$t^n \big(x(t) + y(t)\big).$$

The public key is then written as the sum of a quadratic part and a linear part:

$$P = T \cdot M \cdot (L_1 \otimes L_2) \cdot (z \otimes z) + T \cdot L \cdot S \cdot z.$$

A more compact form is obtained by defining

$$A = M \cdot (L_1 \otimes L_2).$$

With this notation, the public key becomes

$$P = T \cdot (A \mid LS) \cdot (z \otimes z \| z).$$

This last expression is the compact public key representation. It gathers in a single form the quadratic terms coming from the tensor product and the linear terms added by the extra layer.

4.5 Enc & Dec stages

In EDF a solution is guaranteed to exists, but the central map has few surjections that must be eliminated by the verification with the recovered plaintext against a public hash constructed from the original plaintext.

- Bob selects a value $X \in F_p^{2n}$ and a salt s.
- Bob computes the hash value $h = H(X\|s)$.
- Bob computes the ciphertext $y \in F_p^{2n}$ such that $P(X) = y$.
- Bob sends (y, h, s) to Alice.
- Alice computes $z = T^{-1} \cdot y$.
- Alice converts z to a polynomial $\phi(z) = z(t)$ and adds the monic term t^{2n}, obtaining
$$z'(t) = t^{2n} + z(t).$$
- Alice factors $z'(t)$ into irreducible factors over F_p.
- Alice applies the partition extension algorithm and works with λ_{ext} from 2.3.
- Alice applies equal-degree factorisation with target degree $d = n$ by invoking $SS(d, \lambda_{\text{ext}})$.
- Each valid subset-sum solution gives a pair of equal-degree polynomials $z_1(t), z_2(t)$ such that $z_1(t)z_2(t) = z'(t)$.
- For each such pair, Alice computes a candidate preimage
$$X_i = S^{-1} \cdot (z_1\|z_2).$$
- Alice checks whether
$$H(X_i\|s) = h = H(X||s).$$
If the equality holds, then X_i is accepted as the correct recovered value. Otherwise, Alice continues with the next valid subset sum solution.

5 Experimental Testing

5.1 Material

The EDF, Facto-DSA, and X-1 test scripts are provided in [12]. They were written in Python using SageMath, following the same general structure. First, an odd prime p is fixed. Then, all dimension parameters are defined from n. Key generation is implemented with linear algebra and tensor algebra, while polynomial algebra is mainly used later in the factorisation step. The scripts do not directly run the Gröbner basis computation. Instead, they generate the corresponding MAGMA input script.

$$\texttt{facto-algo-}p\texttt{-}m\texttt{-}n,$$

where `algo` denotes the algorithm name.

The dimensions of the output systems are as follows. EDF produces $2n$ equations in $2n$ variables. Facto-DSA produces $2n-1$ equations in $2n$ variables. X-1 also produces $2n-1$ equations in $2n$ variables. In our tests, however, X-1 is not used in the parameter range $m \leq k$ allowed by its authors, that is, we do not consider the case where the matrix T has lower dimension.

Each script validates the trapdoor construction by means of input-output assertions. After that, security is tested experimentally through Gröbner basis computation on the ideal generated by the output equations.

The test process is the following:

- The public key map $P(z) = y$ is generated using tensor algebra.
- A random point z is evaluated in P to obtain the output vector y.
- In the DSA setting, a signature point is generated from y. In the EDF setting, the original z is recovered.
- A MAGMA script is then generated for the ideal defined by $P(z) - y$, both with and without field equations, and stored in a text file of the form `facto-algo-p-m-n.txt` and `facto-algo-feqs-p-m-n.txt` for the field equations $x_i^p - x_i = 0$.

5.2 Measurement scope

The experiments reported here measure only the resources needed to compute a Gröbner basis with the F_4 algorithm. For each instance, we test the original polynomial system and, separately, the same system with the field equations

$$x_1^q - x_1, \ldots, x_n^q - x_n.$$

When reporting a given time or memory value, it means that these resources were required to compute a Gröbner basis of the corresponding ideal with F_4. Therefore, the measurements do not include post-processing steps used to recover solutions in the variety.

The quantities d_{solv} and ff_d describe two different moments in the Gröbner basis computation. The value ff_d, or first fall degree, is the first degree at which a nontrivial reduction appears below the degree of regularity expected for a non-structured MQ system. It is a signal that the system starts to simplify algebraically in a useful way. By contrast, d_{solv} is the solving degree: it is the largest degree reached by the Gröbner basis computation before the system is fully reduced to a form from which the solution can be extracted. One may think of ff_d as the first important drop in degree during the computation, while d_{solv} is the final critical degree that controls the overall algebraic cost. In general, a smaller ff_d suggests that useful relations appear earlier, whereas a larger d_{solv} usually indicates a harder system.

Symbol	Meaning
p	Ground field characteristic
m	Number of equations
n	Number of variables
d_{solv}	Solving Degree
ff_d	First fall degree
F_4	Time spent in F4 alg.
time	Real time
MB	RAM in megabytes

Table 1: Results without field equations.

Algorithm	p	m	n	d_{solv}	ff_d	F_4	time	MB
X-1	7	13	14	12	9	90.070	22.891	1809.2
Facto-DSA	7	13	14	10	9	1349.220	146.514	3677.5
EDF	7	14	14	8	2	76.030	8.688	354.8
X-1	7	15	16	12	10	10537.340	1275.622	7793.5
Facto-DSA	7	15	16	10	10	144279.300	25148.422	19507.0
EDF	7	16	16	9	2	4339.399	372.802	3809.1
X-1	11	13	14	12	9	84.930	22.495	1970.6
Facto-DSA	11	13	14	13	9	1358.029	134.716	1681.7
EDF	11	14	14	8	2	71.050	7.740	351.9
X-1	11	15	16	14	10	7698.319	1380.284	8229.4
Facto-DSA	11	15	16	11	10	189025.559	41459.463	25559.6
EDF	11	16	16	9	2	4392.550	376.495	3956.8

Table 2: Results with field equations.

Algorithm	p	m	n	d_{solv}	ff_d	F_4	time	MB
X-1	7	13	14	8	8	10.259	3.237	390.6
Facto-DSA	7	13	14	8	8	117.930	15.449	824.3
EDF	7	14	14	7	2	19.170	3.162	591.5
X-1	7	15	16	8	8	116.450	23.404	1283.9
Facto-DSA	7	15	16	8	8	1679.220	186.672	3945.3
EDF	7	16	16	7	2	267.740	29.859	1617.0
X-1	11	13	14	12	9	75.510	15.357	1340.3
Facto-DSA	11	13	14	12	9	1706.650	527.256	6488.9
EDF	11	14	14	11	2	116.750	44.038	3916.0
X-1	11	15	16	12	10	5552.840	712.266	6220.7
Facto-DSA	11	15	16	10*	10	70657.650*	64800*	38456.1*
EDF	11	16	16	11	2	5562.869	5951.734	18200.7

* The run `output-facto-DSA-feqs-11-15-16` was interrupted before completion. The row reports observed values only: the computation had already shown a first fall at degree 10, and the reported F_4, time, and memory are non-final observed values at the moment the run was stopped.

5.3 Conclusions

The comparison of the three algorithms is clear in both settings. Without field equations, Facto-DSA is the hardest system overall: it gives the largest F_4 CPU times, the largest elapsed times, and in general the largest memory usage. X-1 is always in the middle, while EDF is the easiest one, with the lowest solving cost in almost all tested instances. This ranking is also compatible with the degree indicators: EDF has the smallest ff_d and low d_{solv}, X-1 is higher, and Facto-DSA leads to the largest global cost even when d_{solv} alone is not always maximal. Thus, in the standard setting, the practical order is

$$\text{EDF} \ < \ \text{X-1} \ < \ \text{Facto-DSA}.$$

With field equations, the same order is still observed at the global level, but the effect depends strongly on the field size. For $p = 7$, field equations improve the three algorithms: F_4 CPU time and elapsed time decrease substantially, and the gain is especially strong for X-1 and Facto-DSA. For $p = 11$, X-1 still benefits, but EDF and Facto-DSA become clearly more expensive in the larger instances, both in elapsed time and in memory. Hence field equations are useful in the small-field regime, but this advantage weakens, and can even reverse, when p grows.

Therefore, the tables support the following reading. Facto-DSA is the strongest construction from the comparison, since it remains the most expensive to solve with or without field equations. X-1 is consistently intermediate. EDF is the easiest one in the standard setting, although with field equations its cost may increase sharply for larger p. Overall, the main practical conclusion is that field equations should be seen as a small-field optimization, not as a uniform improvement for all algorithms and all parameter choices. MAGMA output from the experiments can be find in [12], folder 'analysed-results'.

References

[1] Borja Gómez. *Hidden Irreducible Polynomials: A cryptosystem based on Multivariate Public Key Cryptography.* IACR Cryptology ePrint Archive, Paper 2019/1174, 2019.

[2] Christian Eder. *Breaking the Hidden Irreducible Polynomials Scheme.* IACR Cryptology ePrint Archive, Paper 2019/1283, 2019.

[3] Irene Di Muzio, Martin Feussner, and Igor Semaev. *Multivariate Signatures with Polynomial Factorization.* IACR Cryptology ePrint Archive, Paper 2025/2061, 2025.

[4] Christopher Wolf and Bart Preneel. *Taxonomy of Public Key Schemes based on the Problem of Multivariate Quadratic Equations.* IACR Cryptology ePrint Archive, Paper 2005/077, 2005.

[5] Tsutomu Matsumoto and Hideki Imai. *Public Quadratic Polynomial-Tuples for Efficient Signature-Verification and Message-Encryption.* In *Advances in Cryptology — EUROCRYPT '88*, 1988.

[6] Jacques Patarin. *Hidden Fields Equations (HFE) and Isomorphisms of Polynomials (IP): Two New Families of Asymmetric Algorithms.* Available at: `http://www.minrank.org/hfe.pdf`.

[7] Jean-Charles Faugère. *A New Efficient Algorithm for Computing Gröbner Bases (F4). Journal of Pure and Applied Algebra*, 139(1–3):61–88, 1999.

[8] Jean-Charles Faugère and Antoine Joux. *Algebraic Cryptanalysis of Hidden Field Equation (HFE) Cryptosystems Using Gröbner Bases.* In *Advances in Cryptology — CRYPTO 2003*, 2003.

[9] Aviad Kipnis, Jacques Patarin, and Louis Goubin. *Unbalanced Oil and Vinegar Signature Schemes.* In *Advances in Cryptology — EUROCRYPT '99*, 1999.

[10] J. O. Shallit, G. S. Frandsen, and J. F. Buss. *The Computational Complexity of Some Problems of Linear Algebra.* Available at: `https://www.brics.dk/RS/96/33/BRICS-RS-96-33.pdf`.

[11] "Number of Irreducible Polynomial Factors of a Polynomial in $F_p[X]$," *Mathematics Stack Exchange*, question posted Aug. 6, 2020. Available online.

[12] *Resources for analysing and testing factorisation based schemes* `https://github.com/kub0x/facto-mpkc`